Object Oriented Programming

Dr. Sangita M. Jaybhaye
Dept. of CSE(AI)

Procedural Programming

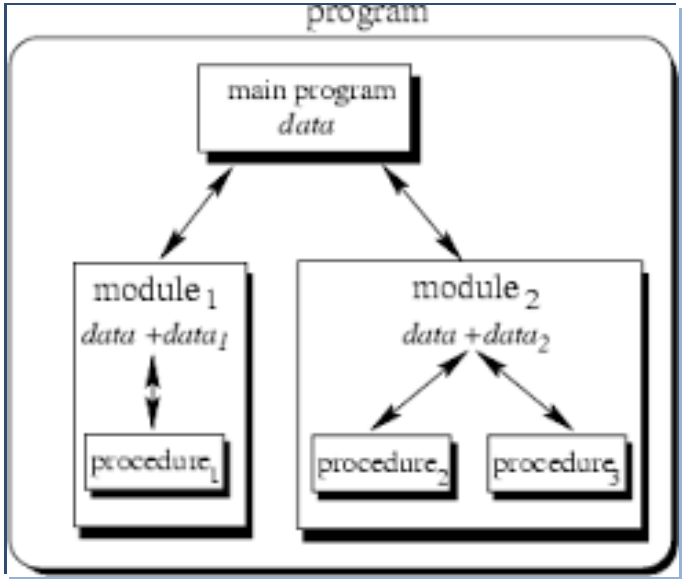
- Programming using high level language such as COBOL,FORTRAN and C is known as procedure oriented programming.
- The main program is divided into number of functions
- Each function has its own local data and it can access global data as well, it is difficult to identify what data is used by which functions.
  - Sequence of Things
  - Emphasis is on Doing Things i.e. Procedures.
  - Share Global Data

- Modular Programming
  - Emphasizes Separating The Functionality
  - Modules Perform Logically Discrete Functions.
  - No Interaction Between Modules.
  - Each Module Works Independently

  Benefits of Modular programming:

  1. Efficient program development

     small subprograms are easier to design
  2. Multiple use of subprograms
  3. Ease of debugging and modifying


program
main program
data
module 1
data +data1
procedure1
module 2
data +data2
procedure2
procedure3

- Generic Programming
  - Generic Programming is about Generalizing Software Components.
  - Algorithms Are Written In Terms Of To-be-specified-later.
  - In C++, Class And Function Templates Are Particularly Effective Mechanisms.

- Object-Oriented Programming
  - Decomposition of problem into a number of entities called objects
  - Emphasis is on data rather than procedure.
  - The same message works differently for various object of data
  - Objects can inherit traits of previously created objects
  - Information hiding is more complete than in procedural programs
  - New data and functions can be added easily whenever necessary.
  - Follows bottom-up approach
  - Data is not accessible to external functions which ensures data security.

- Limitations of procedural programming
  - Difficult to relate with real world objects
  - Abstraction and modularization is difficult
  - Difficult to understand
  - Difficult to maintain

- Need of object-oriented programming
  - Possible to map objects in the problem domain
  - Data Hiding : Security
  - Inheritance : Eliminate redundancy , reusability
  - Its flexibility
  - It makes the coding more organized

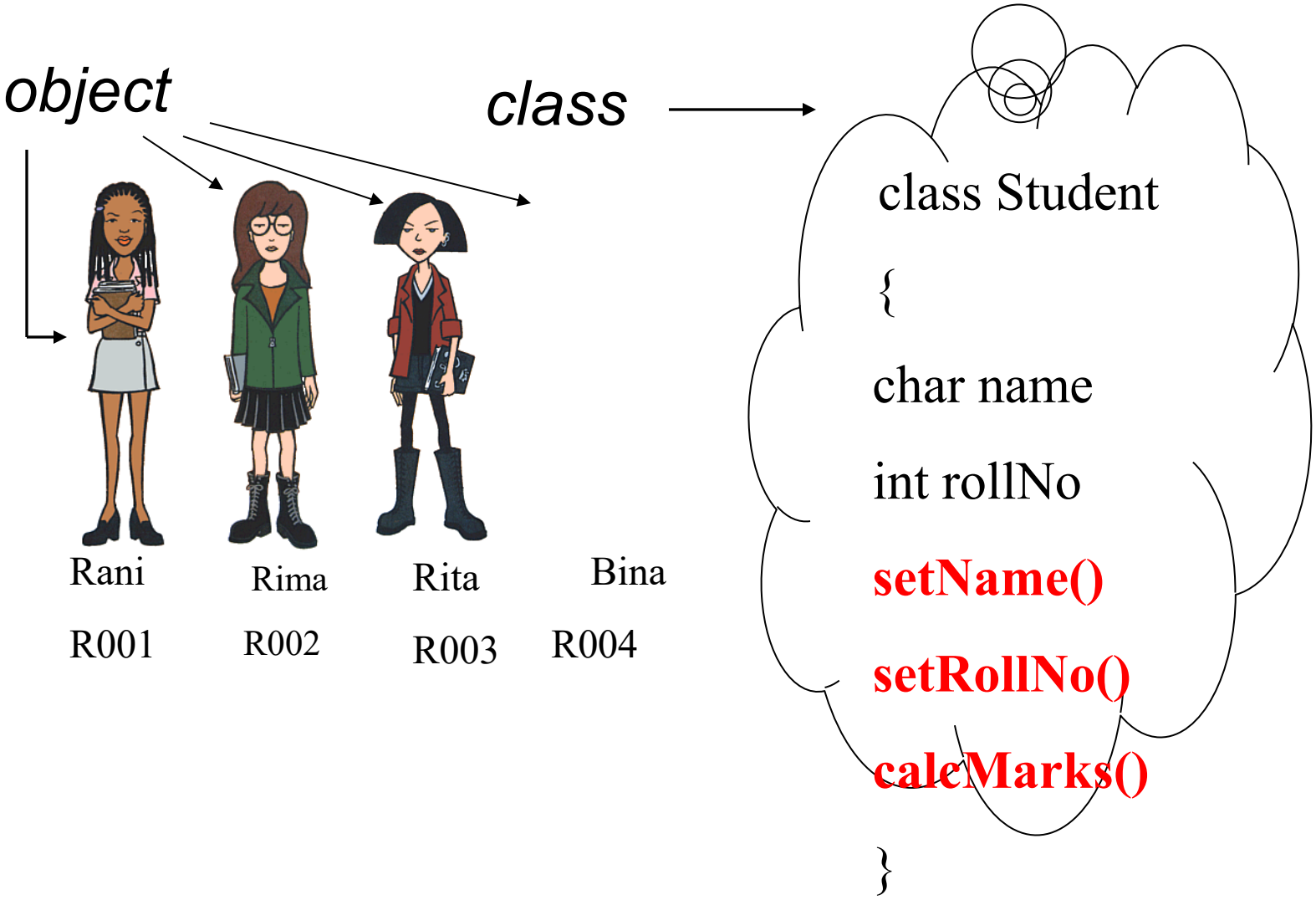
Structured Vs OOP

Structured	OOP
Focuses on Process	Focuses on Object
Follows Top Down Approach	Follows Bottom Up Approach

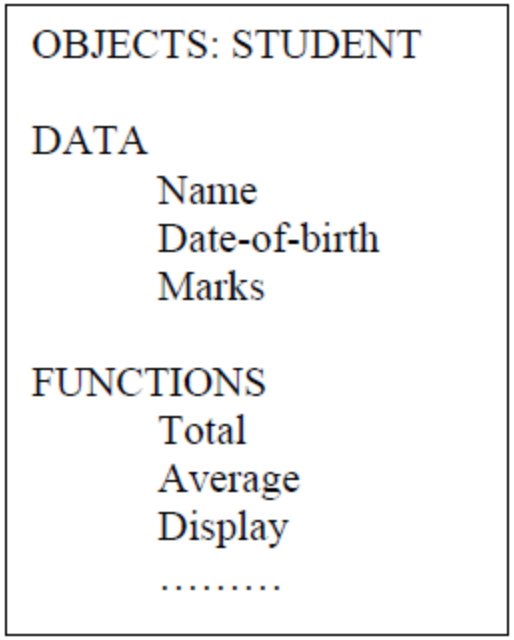
	Procedure Oriented Programming	Object Oriented Programming
Divided Into	In POP, program is divided into small parts called **functions**	In OOP, program is divided into parts called **objects**
Importance	In POP, Importance is not given to **data** but to functions as well as **sequence** of actions to be done	In OOP, Importance is given to the data rather than procedures or functions because it works as a **real world**
Approach	POP follows **Top Down approach**	OOP follows **Bottom Up approach**
Access Specifiers	POP does not have any access specifier.	OOP has access specifiers named Public, Private, Protected, etc.
Data Moving	In POP, Data can move freely from function to function in the system	In OOP, objects can move and communicate with each other through member functions
Expansion	To add new data and function in POP is not so easy	OOP provides an easy way to add new data and function
Data Access	In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system	In OOP, data can not move easily from function to function, it can be kept public or private so we can control the access of data
Data Hiding	POP does not have any proper way for hiding data so it is **less secure**	OOP provides Data Hiding so provides **more security**
Examples	Example of POP are : C, VB, FORTRAN, Pascal	Example of OOP are : C++, JAVA, VB.NET, C#.NET

- Fundamentals Of Object-oriented Programming
  - Objects
  - Classes
  - Data Members
  - Methods
  - Messages
  - Data Encapsulation
  - Data Abstraction And Information Hiding
  - Inheritance
  - Polymorphism

Example : Objects and Classes



- Object
  - Object is created from class.
  - It is exact replica of class definition.
  - Objects are basic run time entities.
  - Object will be allocated memory & hence it has physical existence.
  - Each object take up space in the memory to store data and code.
  - Representation of object



Class and Object

- **Class is a user defined data type, which holds its own data members and member functions, which can be accessed and used by creating instance of that class.**
- **The variables inside class definition are called as data members and the functions are called member functions.**
- **Object is instance of the class.**
- **Class is a collection of objects of similar type.**

e.g. Consider a fruit class

Grapes, Apples, Bananas etc are objects.

Potato is not an object of fruit class.

- We can create multiple objects of same class.

Student s1,s2;

- Both objects will contain all attributes as defined in class.
- Though they will posses all class attributes , the value of these attributes will be different.

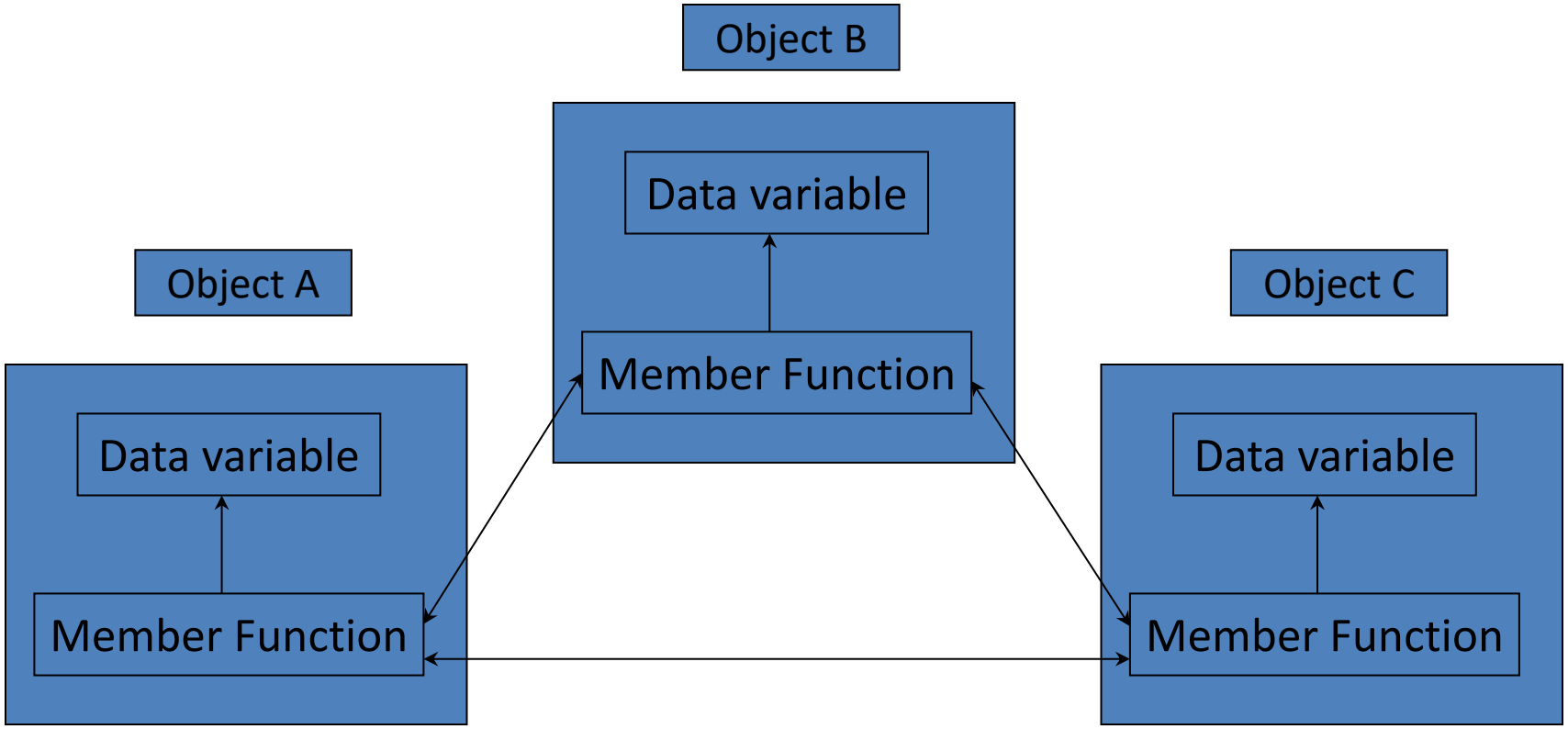
S1.roll=1;

S2.roll=2;

```
class addition
{
      int  a,b,c;                                  //Private Data Members
public:
      void read();                                 //Public member functions
      void add();
};
void addition :: read()
{
      cin>>a>>b;
}
void addition :: add()
{
      c=a+b;
      cout<<"\nThe addition is="<<c<<"\n";
}
int main()
{
      addition ob1,ob2;
      ob1.read();
      ob1.add();
      ob2.read();
      ob2.add();
}
```

- Class
  - **Class is used to create user defined data type.**
  - A class is a blueprint or prototype that defines the data members and the methods (functions) common to all objects of a certain kind.
  - Objects (data and functions) are members of user-defined types called classes.
  - A class definition is an extension of a C structure definition , made up of declarations of variables and of function
  - The **variables are typically declared to be private or protected and the functions are typically declared to be public.**
  - **Structures members are public by default and class members are private by default.**
  - Class only provides template for object & does not create any memory space for object.

- Program is composed of a collection of individual units or *objects*, as opposed to a traditional view in which a program is a list of instructions to the computer.



- Data members
  - The variables are typically declared to be private or protected

- Methods or Member functions

  - Functions are typically declared to be public.
  - Defined outside or inside class.
  - Objects communicate with each other.

POP VS OOP

	Procedural Oriented Programming	Object Oriented Programming
Divided Into	In POP, program is divided into small parts called functions.	In OOP, program is divided into parts called objects.
Importance	In POP, Importance is not given to data but to functions as well as sequence of actions to be done.	In OOP, Importance is given to the data rather than procedures or functions because it works as a real world.
Approach	POP follows Top Down approach.	OOP follows Bottom Up approach.
Access Specifiers	POP does not have any access specifier.	OOP has access specifiers named public, private, protected, etc.
Data Moving	In POP, Data can move freely from function to function in the system.	In OOP, objects can move and communicate with each other through member functions.
Expansion	To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.
Data Access	In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data cannot move easily from function to function. It can be kept public or private so we can control the access of data.
Data Hiding	POP does not have any proper way for hiding data so it is less secure.	OOP provides Data Hiding so provides more security.
Overloading	In POP, Overloading is not possible.	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading.
Examples	Example of POP are: C, VB, FORTRAN, and Pascal.	Example of OOP are: C++, JAVA, VB.NET, C#.NET.

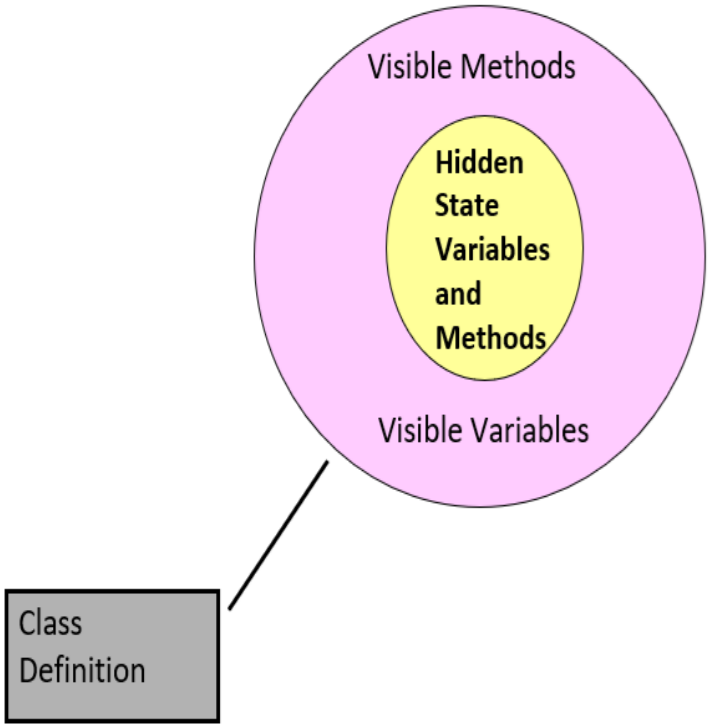
What is Encapsulation?

The wrapping up of data and functions into a single unit is known as encapsulation.

When classes are defined, programmers can specify that certain methods or state variables remain hidden inside the class.

These variables and methods are accessible from within the class, but not accessible outside it.

The insulation of the data from direct access by the program is called data hiding or information hiding.



What is abstraction?

Abstraction refers to the act of representing essential features without including the background details or explanations.

Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight, and functions to operate on these attributes. They encapsulate all the essential properties of the objects that are to be created.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc. in the car. This is what abstraction is.

Significance of using OOP in software development

Increases productivity: OOP allows programmers to construct new programs quickly by using reusable code and multiple libraries.

Improves maintainability: OOP's modularity and reusability reduce redundancy and make code easier to maintain.

Simplifies troubleshooting: OOP's self-contained nature makes it easier to identify the source of a problem in the code.

Models complex systems: OOP allows developers to create intuitive models of complex systems by representing real-world entities as objects with properties and behaviors.

Saves time: OOP's polymorphism allows a single function to adapt to the class it's in, saving time and making it easy to add new vehicle types.

Enhances flexibility: OOP's inheritance and polymorphism provide flexibility for your codebase.

Reduces development costs: OOP's maintenance and reusability benefits reduce development costs.

Standard Library and Namespaces:

- In C++, a **namespace** is a collection of related names or identifiers (functions, class, variables) which helps to separate these identifiers from similar identifiers in other namespaces or the global namespace.
- The identifiers of the C++ standard library are defined in a namespace called std.
- In order to use any identifier belonging to the standard library, we need to specify that it belongs to the std namespace. One way to do this is by using the scope resolution operator ::

- For e.g. `std::cout << "Hello World!";`

- Here, we have used the code std:: before cout. This tells the C++ compiler that the cout object we are using belongs to the std namespace.

C++ std Identifiers

All the standard library identifiers provided by the standard header files like <iostream>, <string>, <vector>, etc. are declared in the std namespace.

For example, identifiers cin and cout are defined inside the standard header file <iostream> of the namespace std.

The first way we access identifiers in the std namespace is by directly qualifying the identifier with the prefix std::. Here,

std is the C++ standard library namespace

Introduction to the C++ Standard Library

The C++ Standard Library is a collection of functions, containers, and other tools that can be used in C++ programs:

Containers: Generic containers for storing data

Functions: Functions for manipulating containers, performing common tasks, and more

Streams: Generic streams for input and output, including interactive streams

Support: Support for some language features

The C++ Standard Template Library (STL) is a set of template classes and functions that provides the implementation of common data structures and algorithms such as lists, stacks, arrays, sorting, searching, etc.

Components of STL

The components of STL are the features provided by Standard Template Library (STL) in C++ that can be classified into 4 types:

Containers

Algorithms

Iterators

Functors

Overview of namespaces in C++,

Namespace provide the space where we can define or declare identifier i.e. variable, method, classes.

Using namespace, you can define the space or context in which identifiers are defined i.e. variable, method, classes. In essence, a namespace defines a scope.

Importance of using namespaces :

Example, you might be writing some code that has a function called xyz() and there is another library available which is also having same function xyz(). Now the compiler has no way of knowing which version of xyz() function you are referring to within your code.

A namespace is designed to overcome this difficulty and is used as additional information to differentiate similar functions, classes, variables etc. with the same name available in different libraries.

The best example of namespace scope is the C++ standard library (std) where all the classes, methods and templates are declared. Hence while writing a C++ program we usually include the directive using namespace std;

structure of C++ program

The structure of the program written in C++ language is as follows:

Documentation Section:

This section comes first and is used to document the logic of the program that the programmer going to code.

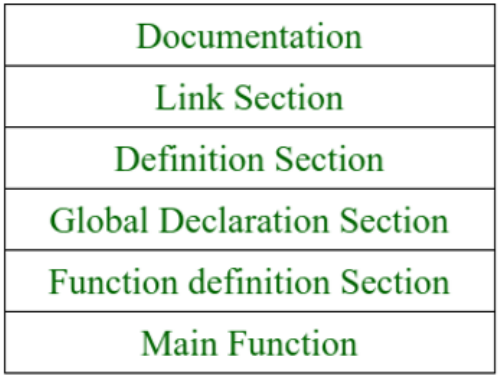
It can be also used to write for purpose of the program.

Linking Section:

The linking section contains two parts:

Header Files:

Standard headers are specified in a program through the preprocessor directive #include.



Skeleton of C Program

Cont..

Namespaces:

A namespace permits grouping of various entities like classes, objects, functions, and various C++ tokens, etc. under a single name.

Namespaces can be accessed in multiple ways:

```
using namespace std;
using std :: cout;
```

Definition Section:

It is used to declare some constants and assign them some value.

Global Declaration Section:

Here, the variables and the class definitions which are going to be used in the program are declared to make them global.

Function Declaration Section:

It contains all the functions which our main functions need.

Usually, this section contains the User-defined functions.

Main Function:

The main function tells the compiler where to start the execution of the program. The execution of the program starts with the main function.

Example of C++ program

```
#include <iostream>

int main() {
    std::cout << "Hello World!";
    return 0;
}
```

Output

```
Hello World!
```

Lab Practice

- Create a class called Book with the following attributes:
- title (string)
- author (string)
- year (int)
- Then create two objects of the class and print their attribute values.

- Class Methods
- Methods are **functions** that belongs to the class.
- There are two ways to define functions that belongs to a class:
- Inside class definition
- Outside class definition

Define a Method Inside the Class

```
class MyClass {          // The class
  public:                // Access specifier
    void myMethod() {  // Method/function defined
inside the class
      cout << "Hello World!";
    }
};

int main() {
  MyClass myObj;     // Create an object of MyClass
  myObj.myMethod();  // Call the method
  return 0;
}
```

Define a Method Outside the Class

```
class MyClass {          // The class
  public:                // Access specifier
    void myMethod();     // Method/function declaration
};

// Method/function definition outside the class
void MyClass::myMethod() {
  cout << "Hello World!";
}

int main() {
  MyClass myObj;      // Create an object of MyClass
  myObj.myMethod();   // Call the method
  return 0;
}
```

Lab practice

- Create a class Student with a method getmarks() and displayresult().
- Then call that method from main() using an object of the class.

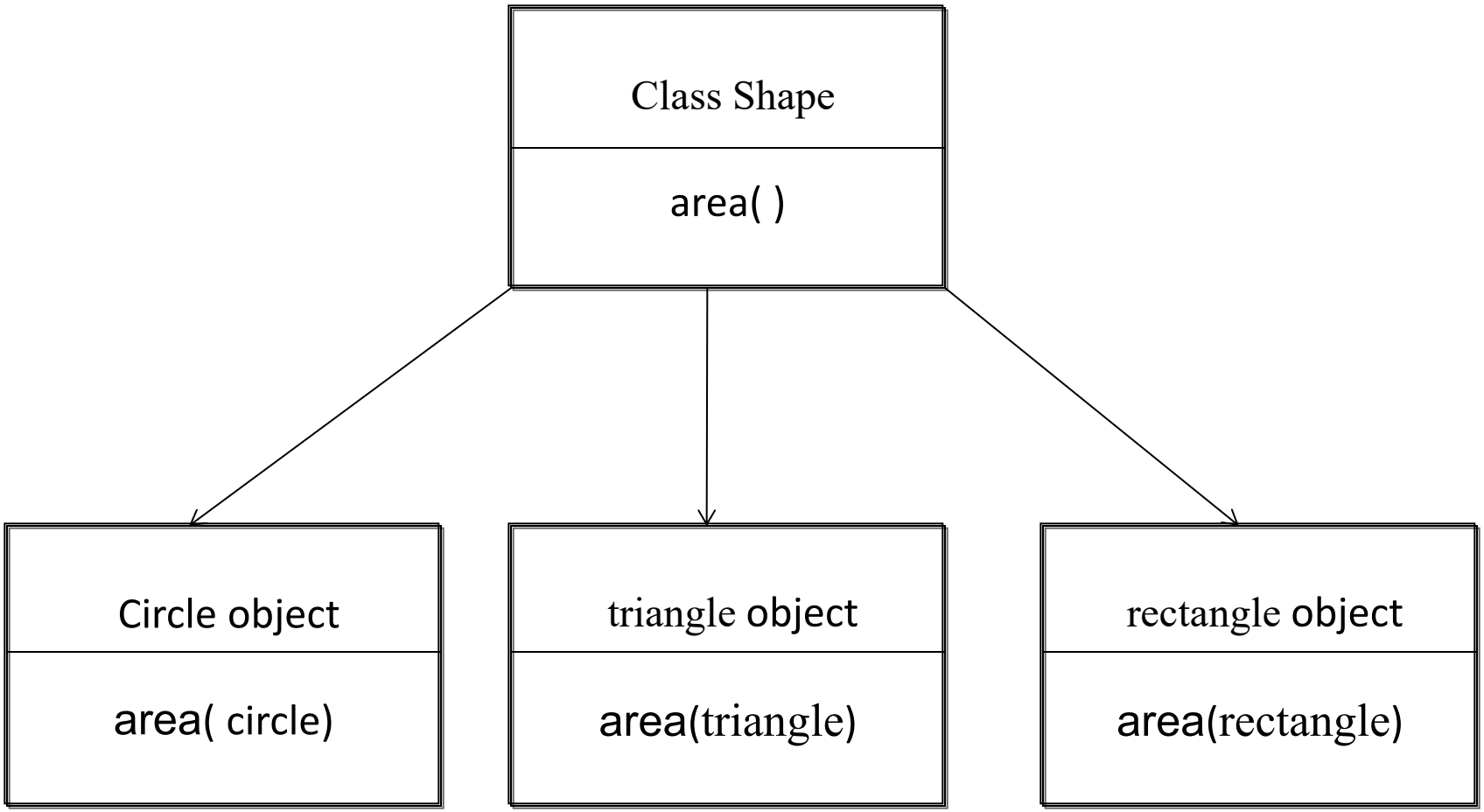
Polymorphism

```
int add (int a, int b);
int add (int a, int b, int c);
float add (int x, float y);
float add (float p, int q);
float add (float x, float y);
```

```
void max(int x, int y)
{
    if(x>y)
        cout<<x<<"is greater";
    else
        cout<<y<<"is greater";
}
void max(float x, float y)
{
    if(x>y)
        cout<<x<<"is greater";
    else
        cout<<y<<"is greater";
}
void max(char x, char  y)
{
    if(x>y)
        cout<<x<<"is greater";
    else
        cout<<y<<"is greater";
```

```
int main()
{
max('A', 'Z');
max(3, 7);
max(44.66, 22.13);

}
```



Virtual function

Encapsulation

- The process of **binding code and data together** in the form of a capsule
- **Data** is wrapped in object is **not accessible to functions outside the class.**
- Functions of the same class can access data & hence this ensures data security.

• Default parameters

- We can provide default values to arguments.
- **Default arguments are always assigned from right to left**
- Examples:
  - int try(int i , int j , int k = 10); //valid
  - int try(int i , int j = 5 , int k = 10); //valid
  - int try(int i = 2 , int j = 5 , int k = 10); //valid

  - int try(int i = 5 , int j , int k); //invalid
  - int try(int i , int j=10 , int k); //invalid
  - int try(int i = 2 , int j = 5 , int k); //invalid

Example

```
void data(int a=10,int b=20)
{
        cout<<a<<b;
}

int main ()
{
data();
data(100);
data(100,200);
}
```

- a = 10 b = 20
- a = 100 b = 20
- a = 100 b = 200

Example

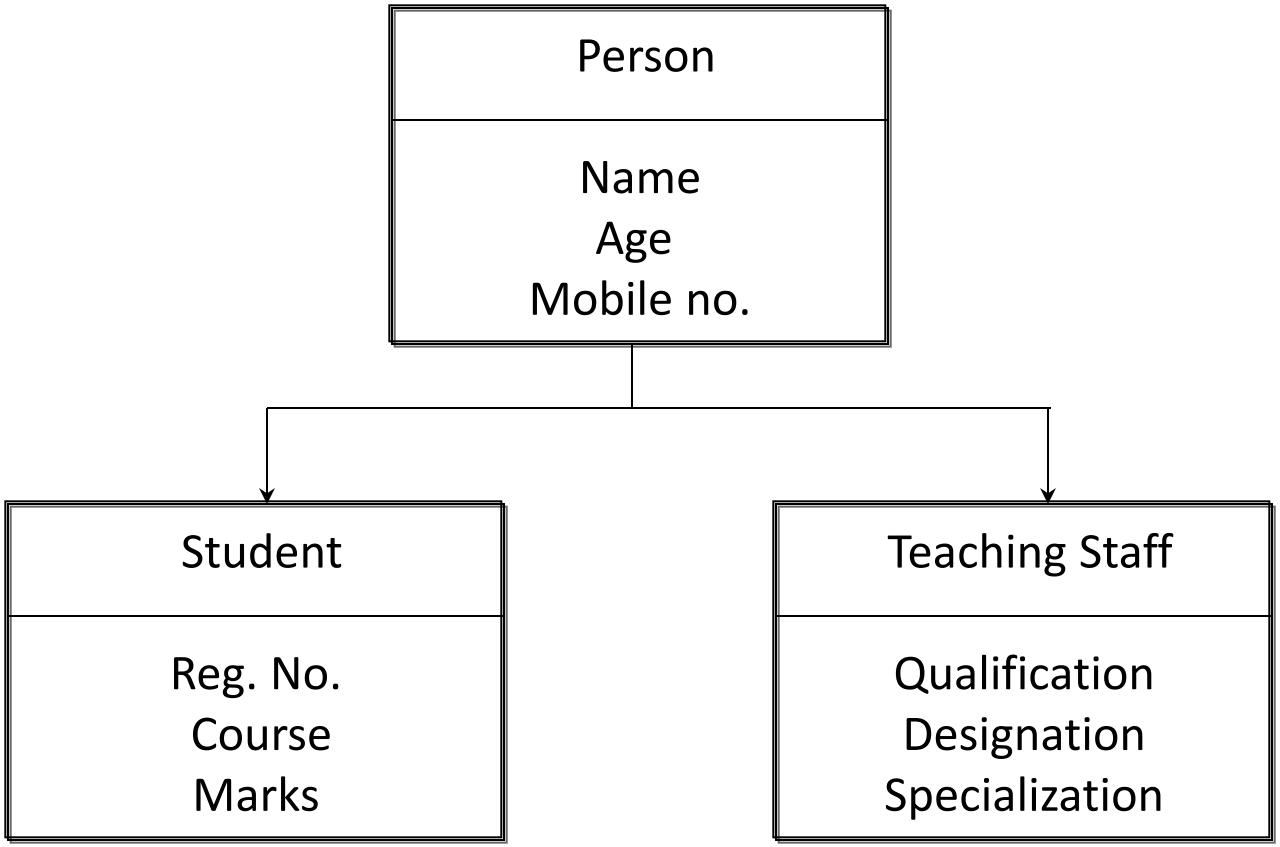
```
void pattern (char M='+', int B=2)
{
cout <<M<<B;
}

void main ()
{
Pattern();
Pattern('*');
Pattern ('#',4)'
Pattern ('$',3);
}
```

Inheritance

- The feature by which one class acquires the properties and functionalities of another class
- Classes are extensible
- You can create new classes that extend or are descendents of existing classes
- The descendent classes can inherit all the attributes of the parent class
- Hierarchical classification
- Reusability

- **Original class is called as Base/super/parent class**
- **Inherited class is called as derived/sub/child class.**



"Person" is a generalization of "Student".
"Student" is a specialization of "Person".

Data Abstraction or Data Hiding

- It hides the detailed implementation of an object from user.
- The process of **extracting the essential information and hiding the irrelevant details** to avoid complexity.
- Example: Money withdraw from ATM

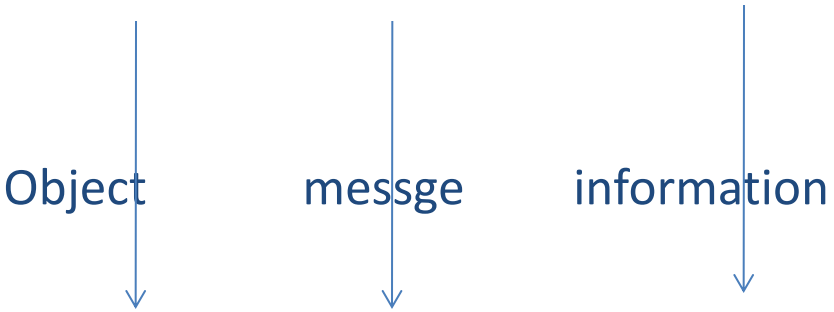
Polymorphism

- Greek word, and the meaning is- many or multiple forms.
- Same operations behave differently in different contexts.

- The operation “area “ is same but it will take different forms depending upon type & number of arguments passed to area.

- Message Passing
  - Object communicate with each other by sending and receiving information
  - Message is nothing but request for execution of a method.

object_name.function_name(arguments)



Message Passing in C++

- **Message Passing** is an OOP concept in which **one object communicates with another by calling its member functions and passing data (messages) as arguments**.

```
#include <iostream>
using namespace std;
class Student {
public:
   void display(string name) {
      cout << "Student Name: " << name << endl;
   }
};
int main() {
   Student s1;
   // Message (name) is passed to the object
   s1.display("XYZ");
   return 0;
}
```

```
Student Name: XYZ
```

One Object Passing a Message to Another Object

```
#include <iostream> using
namespace std;
class Calculator
{
public:
int add(int a, int b)
 { return a + b; }
};
class Result {
public:
void show(int result)
{ cout << "Sum = " << result <<
endl;
}
};
int main()
{
  Calculator c;
  Result d;
  int ans = c.add(10, 20);
 // Message sent to Calculator

   d.show(ans);
      // Message sent to Display
return 0;
}
```

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    double balance;
public:
    BankAccount() {
        balance = 0;
    }
    void deposit(double amount) {
        balance += amount;
        cout << "Amount Deposited: " <<
amount << endl;
    }
    void displayBalance() {
        cout << "Current Balance: " <<
balance << endl;
    }
};
```

```
int main() {
    BankAccount account;

    // Sending messages to the object
    account.deposit(5000);
    account.deposit(2000);
    account.displayBalance();

    return 0;
}
```

```
Amount Deposited: 5000
Amount Deposited: 2000
Current Balance: 7000
```

Reference variables

- alias(alternative name) for previously defined variable.
- Reference variable is created as follows

  Data_type &reference-name = variable-name

Any modification done to existing variable reflects the changes in reference variable & vice versa.

```
#include <iostream>
using namespace std;
int main() {
    int a = 10;
    // Reference variable
    int &ref = a;
    cout << "Value of a = " << a << endl;
    cout << "Value of ref = " << ref << endl;
    ref = 20;   // Changes the value of a
    cout << "After modifying ref:" << endl;
    cout << "Value of a = " << a << endl;
    cout << "Value of ref = " << ref << endl;
    return 0;
}
```

```
Value of a = 10
Value of ref = 10
After modifying ref:
Value of a = 20
Value of ref = 20
```

Comments

- Comments are explanatory statements, ignored by compiler.
- Single line Comments ‘ \\ ’
- Multi line Comments ‘/*’ ‘*/’

Constant arguments

Once the constant is given a value , it cannot be changed during execution of program.

```
int main()
{
      const  int x=10;
      Int y=x+10;
      Cout<<y;
      x++;                                  //Invalid statement
      x--;                                  //Invalid statement
}
```

- Function

  - **Function Declaration/Prototype /signature**

return_type func_name (datatype arg1,datatype arg2,...datatype agrn);

Ex. float volume(int , float, float);

or

float volume(int x, float y, float z); //formal parameters

– Function Definition/ function body

return_type func_name (datatype arg1,datatype arg2,…datatype agrn)

```
{
    function body;
}
```

Ex.

```
float volume(int x, float y, float z)      //formal parameters
{
  function body;
}
```

– Function Call

return_type_var = func_name (arg1,arg2,…. agrn);

Ex. int a=5;
float b=2.5,c=0.5,v;
v = volume(a,b,c); //actual parameters

- **Types of Functions**
  - **No return type, no parameters**

```
void message(void)
    {
       cout<<“Hello World”;
    }
```

  - **No return type but take parameters**

```
void  sum(int a, int b)
    {
       cout<<a+b;
    }
```

  - **Return type and parameters**

```
int sum(int a, int b)
  {
    return a+b;
  }
```

Static Object Creation and Memory Allocation

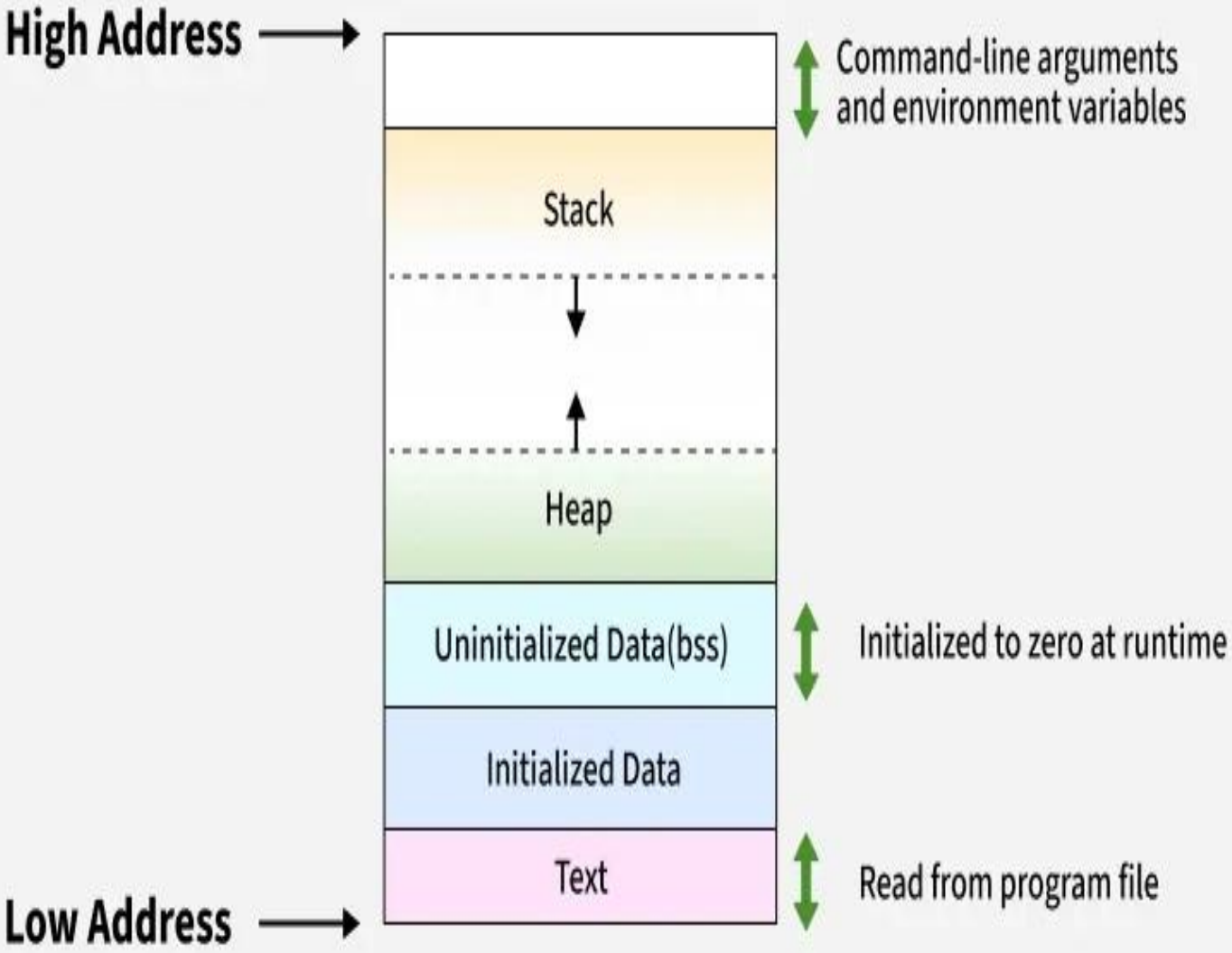
```
#include<iostream>
using namespace std;

class c1
{
    public:
   int a=5;
};

int main()
{
    c1 obj;
    cout<<obj.a;
}
```

Output: 5

Memory Layout of C++ Program



Constructor

- Special member function, it is neither a value-returning function nor a void function.
- There is no need to write any statement to invoke the constructor function. **It is invoked when object of class is created.**
- It is member function of class with same name as that of a class
- It is used to initialize data members of class
- It is always declared under public section
- It does not return any value

```
class test
{
        int x=10;
        int y=20;
};
```

Warning: attempting to initialize a data member of a class explicitly in the class definition is a syntax error.

```
#include <iostream>
using namespace std;

class test
{
public:
   int x = 10;
   int y = 20;
};

int main()
{
   test t;
   cout << "x = " << t.x << endl;
   cout << "y = " << t.y << endl;
   return 0;
}
```

- Contructors can be defined inside a class or outside the class. While defining it outside the class :: operator is used.
- It cannot be declared as static, const & virtual.

Types of constructor

- Default constructor
- Parameterized constructor
- Copy constructor

Types of the Constructors

1. **Default constructors**
   - Default constructor is the constructor which doesn't take any argument
   - It has no parameter
2. **Parameterized constructors**
   - These are the constructors with parameters
   - Using this Constructor we can provide different values to data members of different objects, by passing the appropriate values as argument

3. Copy constructors

- These are special type of Constructors which takes an object as argument,
- Used to copy values of data members of one object into other object

Default constructor

- Constructor with no parameter.
- If constructor is not defined in program then compiler supplies default constructor.

Parameterized constructor

- Constructor that receives parameters.

Copy constructor

- Initializes an object using values of another object passed to it as a parameter.

C++ Constructors

```
class MyClass {        // The class
  public:              // Access specifier
    MyClass() {        // Constructor
      cout << "Hello World!";
    }
};

int main() {
  MyClass myObj;


// Create an object of MyClass (this will call the constructor)
  return 0;
}
```

Parameters

```
#include <iostream>
using namespace std;

class Car {
  public:
    int speed(int maxSpeed);
};

int Car::speed(int maxSpeed) {
  return maxSpeed;
}

int main() {
  Car myObj; // Create an object of Car
  cout << myObj.speed(200); // Call the method with an
argument
  return 0;
}
```

Constructor with Parameters

```
class Car {         // The class
  public:           // Access specifier
    string brand;   // Attribute
    string model;   // Attribute
    int year;       // Attribute
    Car(string x, string y, int z) { // Constructor with parameters
      brand = x;
      model = y;
      year = z;
    }
};

int main() {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
  return 0;
}
```

Constructor Defined Outside the Class

```
class Car {         // The class
  public:           // Access specifier
    string brand;   // Attribute
    string model;   // Attribute
    int year;       // Attribute
    Car(string x, string y, int z); // Constructor declaration
};

// Constructor definition outside the class
Car::Car(string x, string y, int z) {
  brand = x;
  model = y;
  year = z;
}

int main() {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", “XYZ", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " <<
carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " <<
carObj2.year << "\n";
  return 0;
}
```

Lab Practice

- Why Constructors Are Useful ?

Example 1

```
#include <iostream>
using namespace std;
class Z
{
public:
   // constructor
   Z()
   {
      cout<<"Constructor called"<<endl;
   }
   // destructor
   ~Z()
   {
      cout<<"Destructor called"<<endl;
   }
};
```

```
int main()
{
   Z z1;   // Constructor Called
   int a = 1;
   if(a==1)
   {
      Z z2;  // Constructor Called
   }  // Destructor Called for z2
} //  Destructor called for z1
```

- Overloading constructors
  - You can have more than one constructor in a class, as long as each has a different list of arguments.

```
class test
{
 private:
            int a,b;
 public:
        test();                    // constructor
        test(int,int);             // another constructor
        test(float,float);         // another constructor
};
```

Default constructor

```
class test
{
   int x,y;
public:
   test()
   {   x=0;
       y=0;
   }
};
```

Parameterized constructor

```
class test
{       int x,y;
public:
   test(int a, int b)
   {
      x=a;
      y=b;
   }
};
```

Copy constructor

```
class test
{       int x,y;
public:
   test(test &T)
   {
      x=T.x;
      y=T.y;
   }
};
```

```
#include <iostream>
using namespace std;
// Demo class
class A {
public:
   int x;
};
int main()
{
   A a1;
   a1.x = 10;
   cout << "a1's x = " << a1.x << endl;
   // Copy constructor called
   A a2(a1);
   cout << "a2's x = " << a2.x;
   return 0;
}
```

Output

```
a1's x = 10
a2's x = 10
```

```
void test::display()
{
        cout<<x<<y;
}
```

```
void main()
{
test T1;
test T2(10, 20);
test T3(T2);
        T1.display();
        T2.display();
        T3.display();
}
```

Default, copy and Parameterized Constructors

```
class code
{
int id;
public:
code() {}
code(int a) {id = a;}
code(code &x)
{
      id = x.id;
}
void display()
{
      cout << id;
}
}
```

```
int main ()
{
      code A (100);//parameterized
      code B (A); // copy constructor called
      code C = A; // copy constructor called
      code D; // Default constructor

cout << “id of A: ”; A.display();
cout << “id of B: ”; B.display();
cout << “id of C: ”; C.display();
}
```

- Constructor and default parameters

```
rectangle::rectangle(float h=10,float  w=20,int x=100,int y=200)
{
    height = h;
    width = w;
    xpos = x;
    ypos = y;
}

void main()
{
 rectangle rc1();
 rectangle rc2(50);
 rectangle rc3(60,70);
 rectangle rc4(80,90,500);
 rectangle rc5( 100,200,300,400);

}
```

```
#include <iostream>
using namespace std;
class rectangle
{
   float height;
   float width;
   int xpos;
   int ypos;
public:
   // Default constructor
   rectangle()
   {
      height = 10;
      width = 20;
      xpos = 100;
      ypos = 200;
   }
   // Parameterized constructor with default arguments
   rectangle(float h = 10, float w = 20, int x = 100, int y = 200)
   {
      height = h;
      width = w;
      xpos = x;
      ypos = y;
   }
   void display()
   {      cout << "Height = " << height
          << ", Width = " << width
          << ", X = " << xpos
          << ", Y = " << ypos << endl;
   }
```

```
int main()
{
   rectangle rc1;              // Default
constructor
   rectangle rc2(50);
   rectangle rc3(60, 70);
   rectangle rc4(80, 90, 500);
   rectangle rc5(100, 200, 300, 400);
   cout << "rc1: ";
   rc1.display();
   cout << "rc2: ";
   rc2.display();
   cout << "rc3: ";
   rc3.display();
   cout << "rc4: ";
   rc4.display();
   cout << "rc5: ";
   rc5.display();
   return 0;
}
```

- Destructors
  - Special member function which **deletes an object**
  - the same name as the class but is preceded by a tilde (~)
  - **no arguments and return no values**
  - A destructor function is called automatically when the object goes out of scope:
    (1) the function ends
    (2) the program ends
    (3)a *delete* operator is called

Destructors

- ✓ If you do not specify a destructor, the compiler generates a default destructor for you.

- ✓ When a class contains a pointer to memory you allocate, it is your responsibility to **release the memory before the class instance is destroyed.**

```
class test
    {     int x;
    public:
          test()
          {
                x=100;
          }
          ~test()
          {
                cout<<“Object is deleted";
          }
          void display()
          {
                cout<<x;
          }
    };
```

```
void main()
{
    test ob;
    ob.display();
}
```

Example 1: Output

- Constructor called
- Constructor called
- Destructor called
- Destructor called

```
#include <iostream>
using namespace std; class Line
 {
 public:
void setLength( double len );
double getLength( void );
Line();
// This is the constructor
private: double length;
 };
```

Output :

```
// Member functions definitions including constructor
Line::Line(void) {
   cout << "Object is being created" << endl;
}
void Line::setLength( double len ) {
   length = len;
}
double Line::getLength( void ) {
   return length;
}
// Main function for the program int main()
{ Line line;
// set line length line.setLength(6.0);
 cout << "Length of line : " << line.getLength()
 <<endl; return 0; }
```

- Object is being created
- Length of line : 6

```
#include <iostream>
using namespace std;
class Line {
  public:
    void setLength( double len );
    double getLength( void );
    Line(double len);
 // This is the constructor
  private:
   double length;
};
```

```
// Member functions definitions
including constructor
Line::Line( double len) {
  cout << "Object is being created,
length = " << len << endl;
  length = len;
}
void Line::setLength( double len ) {
  length = len;
}
double Line::getLength( void ) {
  return length;
}
```

```
// Main function for the program
int main() {
   Line line(10.0);
    // get initially set length.
   cout << "Length of line : " <<
line.getLength() <<endl;
      // set line length again
   line.setLength(6.0);
   cout << "Length of line : " <<
line.getLength() <<endl;
    return 0;
}
```

- Object is being created, length = 10
- Length of line : 10
- Length of line : 6

```
#include <iostream>

using namespace std;
class Line {
  public:
    void setLength( double len );
    double getLength( void );
    Line();   // This is the constructor declaration
    ~Line();  // This is the destructor: declaration

  private:
    double length;
};
```

```
// Member functions definitions including constructor
Line::Line(void) {
  cout << "Object is being created" << endl;
}
Line::~Line(void) {
  cout << "Object is being deleted" << endl;
}
void Line::setLength( double len ) {
  length = len;
}
double Line::getLength( void ) {
  return length;
}
```

```
// Main function for the program
int main() {
   Line line;
    // set line length
   line.setLength(6.0);
   cout << "Length of line : " <<
line.getLength() <<endl;
    return 0;
}
```

Output:

Object is being created

Length of line : 6

Object is being deleted

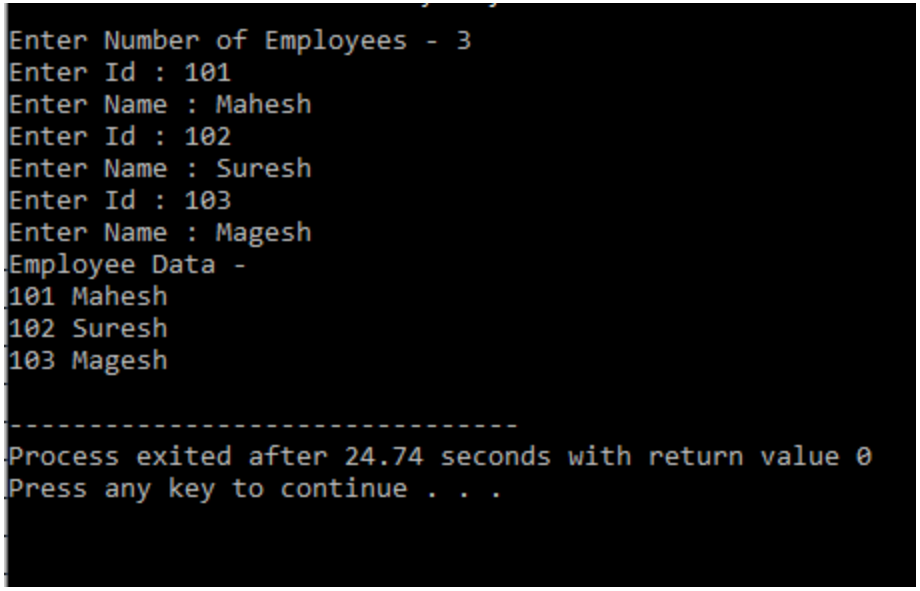
```
// C++ program to implement
// the above approach
#include<iostream>
using namespace std;
class Employee
{
  int id;
  char name[30];
  public:
    // Declaration of function
  void getdata();
    // Declaration of function
  void putdata();
};
// Defining the function outside
// the class
void Employee::getdata()
{
  cout << "Enter Id : ";
  cin >> id;
  cout << "Enter Name : ";
  cin >> name;
}
// Defining the function outside
// the class
void Employee::putdata()
{
  cout << id << " ";
  cout << name << " ";
  cout << endl;
}
```

```
// Driver code
int main()
{
  // This is an array of objects having
  // maximum limit of 30 Employees
  Employee emp[30];
  int n, i;
  cout << "Enter Number of
Employees - ";
  cin >> n;

// Accessing the function
  for(i = 0; i < n; i++)
    emp[i].getdata();

  cout << "Employee Data - " << endl;

  // Accessing the function
  for(i = 0; i < n; i++)
    emp[i].putdata();
}
```


Enter Number of Employees - 3
Enter Id : 101
Enter Name : Mahesh
Enter Id : 102
Enter Name : Suresh
Enter Id : 103
Enter Name : Magesh
Employee Data -
101 Mahesh
102 Suresh
103 Magesh

Process exited after 24.74 seconds with return value 0
Press any key to continue . . .

- **Advantages of Array of Objects:**
- The array of objects represent storing multiple objects in a single name.
- In an array of objects, the data can be accessed randomly by using the index number.
- Reduce the time and memory by storing the data in a single variable.

Disadvantages of Array of Objects:

The main disadvantage with arrays of objects is that the constructor runs for each object. If the default constructor doesn't do the right thing for multiple objects in the array, we have to specify the constructor arguments for each element at the time the array is declared, which we wouldn't have to do if we allocated an array of object pointers, and allocated objects as needed.

Assignment

- 1. Write a C++ program to implement the concept of objects, classes, constructors, destructors.
- 2. Write a program to implement a class for Book details as follows Private: bookid, book_name, author, price Public: get_details(), print_details() Get details of some books from user and print in tabular form. Also find the total prize of all the books.

Dynamic memory allocation

- Compile time:

  Memory is allocated depending upon type & size of data.

- Run time:

  new & delete operators

new operator

pointer_variable= new data_type

```
int *i = new int;
float *j = new float;
char *j = new char;
```

```
int *i = new int[5];
```
allocates memory for an array of 5 integers

```
float *i = new float5];
```
allocates memory for an array of 5 floats

delete operator

- Deallocate memory

```
delete  pointer_variable;

delete  i;
delete  []a;
```

Static memory allocation

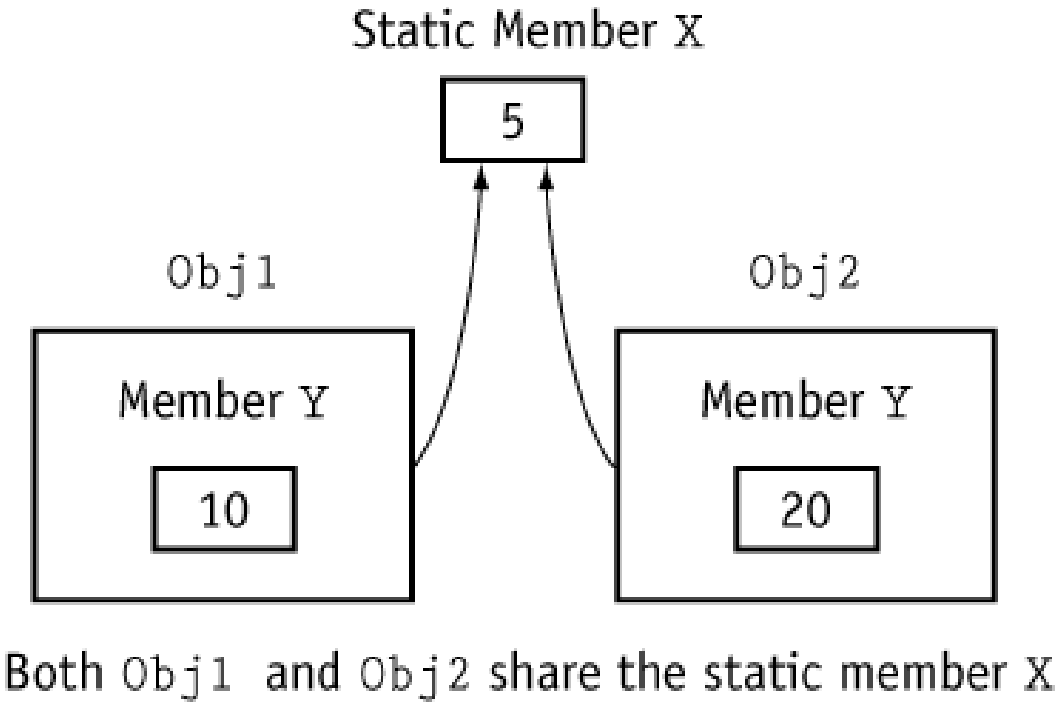
```
void main()
{
        int a[100];
for(i=0;i<5;i++)
        cin>>a[i];
for(i=0;i<5;i++)
        cout<<a[i];
}
```

```
void main()
{
cout<<"enter number of records";
cin>>n;
int *a = new int [n] ;          //Dynamic Memory Allocation
for(i=0;i<n;i++)
        cin>>a[i];
for(i=0;i<n;i++)
        cout<<a[i];
delete [] a;                    //Memory deallocation
}
```

- Static data members
  - Declare inside class with keyword static
  - **It is initialize to zero when the first object of its class is created .**
  - Only one copy of that member is created for the entire class and is shared by all the objects of that class.

Member Variable is Static

int Y;
static int X;



Both Obj1 and Obj2 share the static member X

```
class test
{
    static int count;
    int number;
  public:
    void accept( int n)
    {      number=n;
           count++;
    }
   void display()
    {      cout << number<<count ;
    }
};
```

```
int main ()
{  test ob1, ob2, ob3;
       ob1. display();
       ob2. display();
       ob3. display();
    ob1. accept(100);
    ob2. accept(200);
    ob3. accept(300);
       ob1. display();
       ob2. display();
       ob3. display();
}
```

Count=0

Number=7645

Count=0

Number=29888

Count=0

Number=91645

Count=3

Number=100

Count=3

Number=200

Count=3

Number=300

- Static data members
  - **It is initialize to zero when the first object of its class is created .**
  - Reinitialized at the time of definition outside class

data_type class_name :: static_var_name = value;

```
class test
{
    static int count;
    int number;
  public:
    void accept( int n)
    {      number=n;
           count++;
    }
   void display()
    {      cout << number<<count ;
    }
};
```

```
int test:: count=10;
int main ()
{  test ob1, ob2, ob3;
        ob1. display();
        ob2. display();
        ob3. display();
     ob1. accept(100);
     ob2. accept(200);
     ob3. accept(300);
        ob1. display();
        ob2. display();
        ob3. display();
```

Count=10

Number=7645

Count=10

Number=29888

Count=10

Number=91645

Count=13

Number=100

Count=13

Number=200

Count=13

Number=300

- Static member functions
  - The syntax for a static member function is:

    static <return type> <function name> (<parameter list>);

  - A static member function can have access to only static members declared in the same class.

```
class test
{
    int x;
    static int count;
public :
void accpet()
{
    cin>>x;
}
static void  display ()
{
    cout << count;
    cout<<x;
 }
```

//syntax error

```
int test :: count;

    int main()
    {
        test ob;
         ob.accept ();
        test :: display ();
    }
```

```
class test
{
    int x;
    static int count;
public :
static void  display ()
{
        cout << count;
}
void show()
{       cout << count;
        cout << x;
}
};
```

```
int test :: count=10;
   int main()
    {
        test ob;
        ob.show();
        test :: display ();
    }
```

Static member functions: Example

```
#include < iostream>
class test
{
      int code;
      static int count;
public:
void setcode(void)
{           code=++count;}
void showcode(void)
{           cout<<“object no.:”<<code;}
static void showcount(void)
{           cout<<“count:”<<count; }
};
int test::count;
```

```
int main()
{
test t1,t2;
t1.setcode();
t2.setcode();
test::showcount();
test t3;
t3.setcode();
test::showcount();
t1.showcode();
t2.showcode();
t3.showcode();
return 0;
}
```

Output

Count:2
Count :3
Object number:1
Object number:2
Object number:3

Static Member function

```
class test
{
    int x;
    static int count;
 public :
    void set ()
    {
        x = count++;
    }
    void show ()
    {
        cout << x;
    }
    static void showcount ()
    {
        cout << count ;
    }
};
    int test :: count;
    int main()
    {
        test t1, t2;
        t1.set ();
        t2. set();
        test :: showcount ();
        test t3;
        t3.set();
    }
```

- Class as ADTs and code reuse
  - Abstract data type (ADT): data type that separates the logical properties from the implementation details
  - It is collection of data and associated operations for manipulating that data.

```
int a = 100;
int &b = a;
SOP(a) ;
SOP(b);
        b = b + 10;
                SOP(a) ;
                SOP(b);
        a = 0;
                SOP(a) ;
                SOP(b);
```

- A=100
- B= 100
- A= 110
- B= 110
- A=0
- B=0